

Funkidz Animation

Rapport technique Étape 3 et Étape 4

Notifications e-mail client, animateur et administrateur
Amélioration du panel d'administration des réservations

Projet	Funkidz Animation - application Django
Branche Git	nassim
Date du rapport	17 septembre 2026
Serveur de test	Django runserver, port 8010
Commits	5fab147 (Étape 3) et 945847e (Étape 4)
Suite de tests	46 tests - tous au vert

Workflow suivi

Les deux étapes ont été traitées avec la même séquence, chaque phase devant être réellement terminée avant de passer à la suivante.

1. ANALYSE

Lecture du code existant avant toute modification : configuration e-mail, signaux, modèle Booking, administration, logique d'annulation. Rien n'est supposé, tout est vérifié dans le code.

2. TRAVAIL

Modifications minimales et ciblées. Aucune réécriture du système existant, aucun refactoring général, aucune suppression de code fonctionnel.

3. TESTS

Tests automatisés (manage.py test) et tests réels via HTTP sur le serveur de développement réellement lancé, avec inspection des e-mails produits.

4. VÉRIFICATION

Contrôle des migrations, du système Django, des logs serveur, de l'absence de secret et de la non-régression des étapes précédentes.

5. RAPPORT

Compte rendu : cause, correction, fichiers, tests, ports, procédure de démonstration.

6. COMMIT GIT

git status, git diff, ajout des seuls fichiers de l'étape, commit, vérification du hash et de la propreté du working tree.

Ports et accès nécessaires

Le projet est un monolithe Django : il n'y a pas de frontend séparé à lancer. Les pages sont rendues par des templates Django accompagnés d'Alpine.js et servies par le même processus que l'API REST. Un seul serveur suffit.

Élément	Valeur
Commande backend	venv/bin/python manage.py runserver 8010 --noreload
Port utilisé	8010
Pourquoi pas 8000	Le port Django par défaut 8000 est déjà occupé par un autre service sur la machine de développement (Error: That port is already in use). Aucune configuration du projet n'a été modifiée : le port est un simple argument de lancement. Sur une machine où 8000 est libre, la commande habituelle python manage.py runserver fonctionne sans rien changer.
URL de base	http://127.0.0.1:8010
Frontend	Aucun serveur séparé, aucun port supplémentaire
Compte de test	admin@funkidz.fr (superutilisateur)
SMTP	Aucun port SMTP nécessaire pour les tests : les e-mails sont capturés par le backend console ou le backend fichier. Un port SMTP (587) n'est requis que pour une vraie expédition.

Pages et endpoints utilisés par les tests

URL	Étape	Usage
/auth/login/	3 et 4	Connexion au site
/admin/login/	4	Connexion à l'administration
/reservation/	3	Wizard de réservation
/api/bookings/	3	Création réelle d'une réservation (POST)
/api/contact/	3	Envoi réel d'un message de contact (POST)
/admin/bookings/booking/	4	Liste des réservations, couleurs, tri, liste éditale
/dashboard/	4	Tableau de bord client, bouton d'annulation
/booking/<id>/cancel/	4	Annulation client, règle des 48 heures

Étape 3 - Notifications e-mail

Ce qui existait déjà

Le système d'envoi était déjà en place et fonctionnel : core/emails.py, les templates HTML et texte de templates/emails/, les signaux de bookings/signals.py et availability/signals.py, avec une protection anti-doublon. Il n'a pas été refait.

Ce qui manquait réellement

- Impossible de diriger les notifications administratives vers une boîte de test précise : les destinataires étaient déduits automatiquement et incluaient même l'adresse d'expédition.
- Aucun moyen reproductible de vérifier qui reçoit quoi avant de passer devant le jury.
- Aucune documentation des variables d'environnement à renseigner.

Ce qui a été ajouté

Ajout	Rôle
ADMIN_NOTIFICATION_EMAILS	Variable d'environnement fixant explicitement les destinataires des notifications administratives. Plusieurs adresses possibles, séparées par des virgules. Si elle est vide, le comportement historique est conservé.
EMAIL_FILE_PATH	Dossier de dépôt des e-mails quand on utilise le backend fichier de Django : chaque e-mail devient un fichier lisible.
manage.py demo_email_notifications	Commande qui rejoue le cycle de vie d'une réservation et affiche le destinataire réel de chaque notification.
.env.example	Documentation de toutes les variables e-mail, sans aucun secret.

Qui reçoit quoi

Rôle	Adresse utilisée	Se configure
CLIENT	booking.user.email, le compte qui a réservé	Dans /admin/users/user/
ANIMATEUR	assignment.animateur.user.email, l'animateur affecté	Dans /admin/users/user/
ADMIN	ADMIN_NOTIFICATION_EMAILS, sinon comptes ADMIN et superutilisateurs	Dans le fichier .env

Événements réellement couverts

Événement	Client	Animateur	Admin
Réservation créée (En attente)			oui
Mission attribuée à un animateur		oui	
Réservation confirmée (paiement validé)	oui		oui
Réservation annulée	oui		oui
Échec de paiement	oui		
Message de contact	accusé		oui
Inscription newsletter	oui		
Demande de congé déposée			oui

Événement	Client	Animateur	Admin
Congé approuvé ou refusé		oui	
Rappel la veille de la prestation	oui		

Point non couvert, signalé sans le corriger

Aucune notification n'est envoyée à l'animateur lorsqu'une réservation à laquelle il était affecté est annulée. Hors périmètre de l'étape 3.

Comment tester l'étape 3

A. Vérifier le routage sans rien envoyer

C'est la vérification la plus rapide, à faire juste avant la démonstration. Aucun e-mail n'est expédié : la commande capture les messages en mémoire et contrôle l'adresse de chacun.

```
cd <dossier du projet>
venv/bin/python manage.py demo_email_notifications
```

La commande crée une réservation de démonstration, l'attribue à un animateur, la confirme puis l'annule, affiche le destinataire réel de chaque e-mail avec un verdict OK ou KO, et supprime la réservation à la fin.

B. Démonstration sans compte SMTP (recommandée)

Chaque e-mail est écrit dans un fichier que l'on ouvre devant le jury. Dans le fichier .env :

```
EMAIL_BACKEND=django.core.mail.backends.filebased.EmailBackend
EMAIL_FILE_PATH=sent_emails
ADMIN_NOTIFICATION_EMAILS=<adresse admin choisie>
```

Puis :

```
venv/bin/python manage.py demo_email_notifications --send
ls sent_emails/          # un fichier par e-mail, en-tête To: lisible
```

C. Démonstration avec réception réelle dans trois boîtes

- Créer un compte client avec la première adresse dans /admin/users/user/.
- Créer un compte animateur avec la deuxième adresse, puis son profil animateur.
- Mettre la troisième adresse dans ADMIN_NOTIFICATION_EMAILS.
- Renseigner un SMTP réel dans le .env : EMAIL_HOST, EMAIL_PORT, EMAIL_HOST_USER et EMAIL_HOST_PASSWORD.

```
venv/bin/python manage.py demo_email_notifications --send \
  --client-email <adresse client> \
  --animateur-email <adresse animateur>
```

D. Séquence montrée par la commande

Étape	Action	Notification	Destinataire
1	Création de la réservation	Nouvelle réservation	ADMIN
2	Attribution de la mission	Nouvelle mission d'animation	ANIMATEUR

Étape	Action	Notification	Destinataire
3	Confirmation (paiement validé)	Confirmation et paiement confirmé	CLIENT et ADMIN
4	Annulation	Annulation	CLIENT et ADMIN

La réservation de démonstration est supprimée automatiquement à la fin. L'option --keep permet de la conserver.

Résultats des tests de l'étape 3

#	Action réelle	Destinataire constaté	Verdict
1	POST /api/bookings/ depuis le parcours client	ADMIN : adresse configurée, sujet Nouvelle réservation #16	OK
2	Attribution d'un animateur	ANIMATEUR : animateur@funkidz.fr, sujet Nouvelle mission #16	OK
3	Passage au statut Confirmée	CLIENT : adresse du compte, sujet Confirmation #16	OK
4	Passage au statut Confirmée	ADMIN : adresse configurée, sujet Paiement confirmé #16	OK
5	Passage au statut Annulée	CLIENT puis ADMIN, deux e-mails distincts	OK
6	POST /api/contact/	ADMIN puis accusé de réception au visiteur	OK
7	Contrôle CC et BCC sur tous les e-mails	8 e-mails produits, aucun CC, aucun BCC	OK
8	Logs serveur	0 traceback, 0 erreur 5xx	OK
9	Suite automatisée	Ran 36 tests - OK, dont 3 ajoutés	OK

Sécurité

- Aucun mot de passe SMTP dans le code : EMAIL_HOST_PASSWORD vient uniquement de l'environnement, et un test automatisé le contrôle.
- Le fichier .env n'est pas suivi par Git et n'a pas été modifié.
- .env.example ne contient que des valeurs d'exemple et des avertissements.
- Aucune adresse personnelle codée en dur : les tests utilisent des comptes déjà présents et les domaines example.test et exemple.invalid, réservés aux tests.
- Le dossier sent_emails/ est ignoré par Git : les e-mails de démonstration contiennent des données client.

Commit de l'étape 3

Élément	Valeur
Hash	5fab147
Message	feat: notifications email client animateur admin
Contenu	6 fichiers, 454 insertions, 3 suppressions
Fichiers	core/settings.py, core/utls.py, .env.example, .gitignore, bookings/management/commands/demo_email_notifications.py, bookings/test_email_notifications.py

Étape 4 - Panel d'administration des réservations

Les quatre objectifs

- Rendre les statuts immédiatement lisibles dans la liste des réservations.
- Associer une couleur à chaque statut : vert, orange, rouge.
- Permettre un tri des réservations par date et par heure.
- Appliquer la règle d'annulation client des 48 heures.

Statuts du projet

Les quatre statuts existants n'ont pas été modifiés et aucun nouveau statut n'a été créé.

Valeur en base	Libellé affiché	Pastille	Couleur du texte
CONFIRMED	Confirmée	vert	#15803d
PENDING	En attente	orange	#b45309
CANCELLED	Annulée	rouge	#b91c1c
DONE	Terminée	gris	#334155

Distinguer une annulation admin d'une annulation client

Le modèle ne le permettait pas : il n'existe qu'un seul statut CANCELLED. Plutôt que d'ajouter deux statuts d'annulation, qui auraient cassé tous les tests, signaux, templates et appels API comparant le statut à CANCELLED, un champ séparé cancelled_by (ADMIN ou CLIENT, vide par défaut) a été ajouté. La pastille rouge affiche alors Annulée par administrateur ou Annulée par client.

Tri des réservations

Élément	Valeur réelle dans le projet
Champ date	Booking.booking_date, de type DateField
Champ heure	Booking.booking_time, de type TimeField
Ordre par défaut dans l'admin	booking_date puis booking_time, ordre chronologique croissant
Tri manuel	Les colonnes date, heure et statut restent triables par un clic sur leur en-tête
Navigation par date	date_hierarchy sur booking_date : barre année, mois, jour en haut de la liste
Ordre du modèle	Volontairement inchangé, pour ne pas modifier l'ordre vu par l'API et par la logique de l'étape 2

Aucun champ n'a été créé pour le tri : les champs existants suffisent.

Règle des 48 heures

Ce qui existait avant

Rien. La vue d'annulation client tenait en quatre lignes et ne vérifiait aucun délai : un client pouvait annuler une prestation prévue dans une heure, ou même déjà passée. Aucune mention de 48 heures dans le code, le README ou les tests.

```
booking = get_object_or_404(Booking, id=booking_id, user=request.user)
if booking.status != 'DONE':
    booking.status = 'CANCELLED'
    booking.save()
return redirect('dashboard')
```

Règle désormais appliquée

Le client peut annuler lui-même tant qu'il reste au moins 48 heures avant le début de la prestation. Passé ce délai, l'annulation relève de l'administration, qui n'est soumise à aucun délai.

Situation	Comportement
Plus de 48 heures avant	Annulation acceptée, statut Annulée, origine CLIENT, message de succès, e-mail d'annulation envoyé
Moins de 48 heures avant	Refus, statut inchangé, message invitant à contacter l'agence
Prestation déjà passée	Refus
Réservation déjà terminée	Refus, message dédié
Réservation déjà annulée	Message d'information, aucune action

Le calcul combine `booking_date` et `booking_time` en date et heure complètes, dans le fuseau du projet (Europe/Paris), et les compare à l'instant présent. Le seuil est une constante unique : `Booking.CLIENT_CANCELLATION_DEADLINE_HOURS = 48`.

Sur le tableau de bord, le bouton Annuler est remplacé par un bouton grisé Annulation fermée avec une infobulle explicative dès que le délai est dépassé.

Comment tester l'étape 4

A. Lancer le serveur

```
cd <dossier du projet>
venv/bin/python manage.py migrate          # applique la migration 0005
venv/bin/python manage.py runserver 8010
```

B. Couleurs et tri, dans l'administration

- Ouvrir `http://127.0.0.1:8010/admin/bookings/booking/` et se connecter.
- La colonne Statut affiche une pastille colorée : vert pour Confirmée, orange pour En attente, rouge pour Annulée, gris pour Terminée.
- La liste est triée par date puis par heure. Cliquer sur l'en-tête d'une colonne change le tri ; la barre de dates en haut filtre par année, mois ou jour.

C. Annulation par l'administrateur

- Dans la liste, changer le statut d'une ligne en Annulée puis cliquer sur Enregistrer.
- La pastille passe au rouge et indique Annulée par administrateur.
- Recommencer sur plusieurs lignes à la fois : l'enregistrement fonctionne également, sans l'erreur `booking_time`

corrigée à l'étape 1.

D. Annulation par le client, règle des 48 heures

- Se connecter côté site puis ouvrir `http://127.0.0.1:8010/dashboard/`.
- Cas A : sur une réservation prévue dans plus de 48 heures, le bouton Annuler est actif. Un clic annule la réservation et affiche un message de confirmation.
- Cas B : sur une réservation prévue dans moins de 48 heures, le bouton est remplacé par Annulation fermée. Forcer l'URL `/booking/<id>/cancel/` ne change rien : le statut reste inchangé et un message d'erreur explique le refus.

E. Suite automatisée

```
venv/bin/python manage.py test
# résultat attendu : Ran 46 tests - OK
```

Résultats des tests de l'étape 4

#	Scénario	Résultat obtenu	Verdict
1	Affichage de la liste des réservations	HTTP 200, pastilles présentes, aucune erreur booking_time	OK
2	Réservation confirmée	Couleur #15803d et pastille verte présentes	OK
3	Réservation en attente	Couleur #b45309 et pastille orange présentes	OK
4	Annulation d'une ligne depuis la liste éditabile	HTTP 302, statut Annulée, origine ADMIN, rouge affiché	OK
5	Annulation de plusieurs lignes	HTTP 302, les deux réservations annulées, origine ADMIN	OK
6	Tri par date	Ordre affiché identique à <code>order_by(booking_date, booking_time)</code>	OK
7	Tri par heure	Une réservation à 09:00 passe bien avant une autre à 14:00 le même jour	OK
8A	Annulation client, prestation dans 72 heures	Acceptée, statut Annulée, origine CLIENT	OK
8B	Annulation client, prestation dans 24 heures	Refusée, statut inchangé, origine vide	OK
8C	Tableau de bord dans ce cas	Bouton Annulation fermée affiché	OK
9	Logs serveur	37 requêtes, 0 traceback, 0 erreur 4xx ou 5xx	OK
10	Suite automatisée	Ran 46 tests - OK, dont 12 ajoutés	OK

Migration

Migration additive, aucune donnée touchée

`bookings/migrations/0005_booking_cancelled_by.py` contient un seul `AddField`.
 Les 5 réservations existantes ont reçu une valeur vide et n'ont pas bougé.
 Aucune suppression, aucune modification de champ existant.

Commit de l'étape 4

Élément	Valeur
Hash	945847e
Message	feat: amelioration panel admin reservations
Contenu	6 fichiers, 359 insertions, 8 suppressions
Fichiers	bookings/admin.py, bookings/models.py, bookings/tests.py, bookings/migrations/0005_booking_cancelled_by.py, templates/dashboard/client.html, web/views_client.py

Synthèse

	Étape 3	Étape 4
Objet	Notifications e-mail	Panel admin et annulation
Fichiers modifiés	6	6
Migration	Aucune	1, additive
Tests ajoutés	3	12
Suite complète	36 tests - OK	46 tests - OK
Erreurs serveur	0	0
Régression	Aucune	Aucune
Commit	5fab147	945847e

Régressions vérifiées

- Étape 1 : la modification de statut depuis la liste éditable fonctionne toujours, sans erreur booking_time. Testée à nouveau en réel et couverte par un test dédié.
- Étape 2 : les tests de disponibilité par animateur passent sans modification ; la fonction is_slot_available_for_booking n'a pas été touchée.
- Étape 3 : les tests e-mail passent toujours après l'étape 4 ; les e-mails d'annulation partent normalement.
- Suite complète au vert à la fin de chaque étape.

Points signalés sans être corrigés

- Aucune notification animateur lors de l'annulation d'une réservation à laquelle il était affecté.
- L'annulation client se déclenche par une requête GET sur un lien. Un POST avec jeton CSRF serait préférable, mais la correction dépasse le périmètre de l'étape 4.
- core/settings.py contient une clé de secours pour SECRET_KEY. C'est le comportement d'origine du projet ; en production la variable d'environnement doit être renseignée.

Rappel des commandes utiles

```
# lancer le serveur
venv/bin/python manage.py runserver 8010

# verifier le routage des e-mails sans rien envoyer
venv/bin/python manage.py demo_email_notifications

# demonstration avec envoi reel
venv/bin/python manage.py demo_email_notifications --send

# diagnostic de la configuration SMTP
venv/bin/python manage.py test_email_notifications

# suite de tests complete
venv/bin/python manage.py test
```